

Introducción a la programación.

M. en C. Saul Villegas.

Universidad Autónoma de la Ciudad de México.

26 de abril de 2026

Al finalizar este bloque, el estudiante será capaz de:

- Comprender qué es una función.
- Crear funciones propias.
- Diferenciar variables locales y globales.
- Comprender el paso de parámetros.
- Utilizar punteros para modificar variables.
- Aplicar recursividad.
- Separar programas en módulos.

¿Por qué existen las funciones?

Cuando un programa crece

- Repetir código genera errores.
- Cuesta mantenerlo.
- Es difícil entenderlo.

Solución

Usar funciones permite dividir problemas grandes en partes pequeñas, claras y reutilizables.

¿Qué es una función?

- Es un bloque de instrucciones.
- Realiza una tarea específica.
- Puede recibir datos.
- Puede regresar un resultado.

Ejemplo en C

```
int sumar(int a, int b){  
    return a + b;  
}
```

Ventajas de usar funciones

- Reutilización de código.
- Programas más limpios.
- Más fáciles de corregir.
- Facilitan el trabajo en equipo.
- Reducen el tiempo de desarrollo.

Anatomía de una función

Estructura general

```
tipo nombre(parametros){  
    instrucciones;  
    return valor;  
}
```

Ejemplo

```
float promedio(float a, float b){  
    return (a + b) / 2;  
}
```

Partes

- Tipo de retorno.
- Nombre.
- Parámetros.
- Cuerpo.
- Retorno.

Prototipo, definición y llamada

Prototipo

```
int sumar(int a, int b);
```

Definición

```
int sumar(int a, int b){  
    return a + b;  
}
```

Llamada

```
r = sumar(3, 5);
```

Primero se declara, luego se define y finalmente se utiliza.

Paso de parámetros y punteros

1. Paso por valor

```
void cambiar(int x){  
    x = 100;  
}  
int a = 5; cambiar(a);
```

Resultado: a = 5. No cambia.

2. Para modificar la original

```
int a = 5, *p = &a;
```

- a: valor.
- &a: dirección.
- p: guarda dirección.
- *p: accese al contenido.

3. Paso tipo referencia

```
void cambiar(int *x){  
    *x = 100;  
}  
int main(){  
    int a = 5; cambiar(&a);  
}
```

Ahora: a = 100.

Variables locales y globales

Locales

Solo existen dentro de la función.

```
void f(){  
    int x = 5;  
}
```

Globales

Se declaran fuera de las funciones.

```
int contador = 0;
```

Pueden usarse en varias funciones.

Usar preferentemente variables locales

- Reducen efectos secundarios.
- Facilitan pruebas y depuración.
- Hacen más claro el comportamiento de cada función.

Evitar exceso de globales

- Generan errores ocultos.
- Dificultan el mantenimiento.
- Complican el trabajo en equipo.

Visualización del stack

EL STACK EN LLAMADAS A FUNCIONES EN C

Reiniciar

Tema

1) Sin parámetros
sin retorno

2) Con parámetros
(valor)

3) Con parámetros
(referencia)

4) Con retorno
de valor

5) Recursividad
(factorial)

CÓDIGO

```
1 #include <stdio.h>
2
3 void saludar() {
4     printf("Hola desde saludar()\n");
5 }
6
7 int main() {
8     printf("Antes de llamar a saludar()\n");
9     saludar(); // llamada a función
10    printf("Después de llamar a saludar()\n");
11    return 0;
12 }
```

Descripción del ejemplo

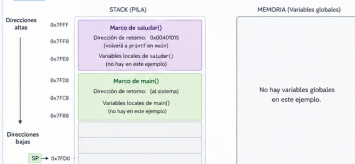
Ejemplo 1: Sin parámetros y sin retorno.

La función saludar() no recibe parámetros y no devuelve ningún valor.

VISUALIZACIÓN DEL STACK

Paso 2/6

Ejecutando: saludar()



Ejecutar paso a paso

Ejecutar hasta el final

Retrosceder paso

SECUENCIA DE EJECUCIÓN



5) RECURSIVIDAD: EJEMPLO factorial(n)

```
1 int factorial(int n) {
2     if (n <= 1) return 1; // caso base
3     return n * factorial(n - 1); // caso recursivo
4 }
5
6 int main() {
7     int r = factorial(4);
8     printf("Resultado: %d\n", r);
9     return 0;
10 }
```

En cada llamada recursiva se crea un nuevo marco en el stack. Al llegar al caso base ($n \leq 1$), comienzan los retornos, desapilando marcos y calculando los resultados.

EJECUCIÓN PARA factorial(4)



Idea clave: El stack permite que las funciones se llamen entre sí, se pasen datos y regresen al punto exacto de donde fueron llamadas.

¿CÓMO USAR?

1. Selección un ejemplo (pestañas de arriba).
2. Presiona "Ejecutar paso a paso" para ver cómo el stack cambia con cada llamada/retorno.
3. Observa el stack (pila) a la derecha.
4. Lee la explicación de lo que ocurre en cada paso.

EXPLICACIÓN DEL PASO ACTUAL

Paso 2: Llamada a saludar()

Cuando main() llama a saludar():

- Se crea un nuevo marco (stack frame) para saludar().
- Se guarda en ese marco la dirección de retorno (0x00401015), que es la siguiente instrucción en main() después de la llamada.
- El Stack Pointer (SP) se mueve hacia abajo (a direcciones menores).
- La ejecución salta al código de saludar().

COMPONENTES DE UN MARCO (STACK FRAME)



CONCEPTOS CLAVE

El stack es LIFO (Last In, First Out)
El último marco que entra es el primero que sale.

El SP (Stack Pointer) siempre apunta al tope del stack.

Cada llamada a función tiene su propio marco independiente.

Al retornar, el marco se elimina y la ejecución continúa en la dirección de retorno guardada.

Caso de estudio: enfriamiento de Newton

Descripción

Se desarrollará un programa en lenguaje C para simular el enfriamiento de un objeto caliente mediante la ley de enfriamiento de Newton.

- Calcula la evolución de la temperatura en pasos discretos.
- Muestra resultados en consola.
- Genera datos para graficarlos posteriormente.

Aplicaciones

Motor, pieza metálica, resistencia eléctrica, horno o una bebida caliente expuesta al ambiente.

Ley de enfriamiento de Newton

$$\frac{dT}{dt} = -k(T - T_a)$$

- $T(t)$: temperatura del objeto.
- T_a : temperatura ambiente.
- k : coeficiente de enfriamiento.
- t : tiempo.

Ecuación discreta

$$T_{n+1} = T_n - k(T_n - T_a)$$

- T_n : temperatura en el paso actual.
- T_{n+1} : temperatura en el siguiente paso.
- T_a : temperatura ambiente.
- k : coeficiente de enfriamiento discreto.
- n : número de iteración.

Objetivo del programa

El programa debe permitir:

- Ingresar temperatura inicial.
- Ingresar temperatura ambiente.
- Ingresar coeficiente de enfriamiento.
- Ingresar número de pasos.
- Calcular la temperatura paso a paso.
- Mostrar una gráfica simple en consola.
- Exportar los datos a un archivo .csv.
- Comparar simulación iterativa y recursiva.

Variables principales

Declaración

```
double T0;  
double Ta;  
double k;  
int pasos;
```

Significado

- T0: temperatura inicial del objeto.
- Ta: temperatura ambiente.
- k: coeficiente de enfriamiento.
- pasos: número de iteraciones.

Función: siguiente temperatura

Prototipo

```
double calcularSiguienteTemperatura(double T,  
                                   double Ta,  
                                   double k);
```

- Recibe la temperatura actual, la ambiente y el coeficiente.
- Devuelve la temperatura del siguiente paso.

Ecuación usada

$$T_{n+1} = T_n - k(T_n - T_a)$$

Actualización por referencia

Prototipo

```
void actualizarTemperatura(double *T,  
                           double Ta,  
                           double k);
```

- Recibe la dirección de memoria de la temperatura.
- Modifica directamente su valor.
- Permite explicar punteros y paso por referencia.

Actualización interna

$$T \leftarrow T - k(T - T_a)$$

Mostrar estado y gráfica en consola

Mostrar estado

```
void mostrarEstado(int paso, double T);
```

Ejemplo:

```
Paso 5 | Temperatura: 67.42 C
```

Gráfica en consola

```
void graficarTemperatura(double T);
```

```
Paso 0 | 90.00 C | #####
```

```
Paso 1 | 84.80 C | #####
```

```
Paso 2 | 80.02 C | #####
```

Prototipo

```
double simularRecursivo(double T,  
                        double Ta,  
                        double k,  
                        int pasos);
```

- Caso base: si $\text{pasos} == 0$, la temperatura final es T .
- Caso recursivo: calcula el siguiente valor y llama otra vez.

Llamada recursiva en código

```
return simularRecursivo(  
    calcularSiguienteTemperatura(T, Ta, k),  
    Ta,  
    k,  
    pasos - 1  
);
```

Idea clave

Cada llamada crea un nuevo frame en el stack hasta llegar al caso base. Después, las llamadas regresan en orden inverso.

Flujo general del programa

Inicio

Pedir T_0

Pedir T_a

Pedir k

Pedir pasos

Mostrar menu

1. Simulacion paso a paso

2. Simulacion recursiva

3. Exportar datos CSV

4. Salir

Fin

Ejemplo numérico

Datos iniciales

$$T_0 = 90\text{ }^{\circ}\text{C}, \quad T_a = 25\text{ }^{\circ}\text{C}, \quad k = 0,08, \quad \text{pasos} = 20$$

Primera iteración

$$T_1 = 90 - 0,08(90 - 25) = 84,8$$

Segunda iteración

$$T_2 = 84,8 - 0,08(84,8 - 25) = 80,016$$

Qué debe observarse en la gráfica

- La temperatura disminuye rápido al inicio porque $T_n - T_a$ es grande.
- Al acercarse a la temperatura ambiente, el enfriamiento se vuelve más lento.
- El sistema tiende a la temperatura ambiente.

Límite esperado

$$\lim_{n \rightarrow \infty} T_n = T_a$$

Conceptos de programación involucrados

- Variables
- Condicionales
- Ciclos
- Funciones con retorno
- Funciones void
- Paso por valor
- Paso por referencia
- Punteros
- Recursividad
- Stack y archivos CSV

Paso por valor

```
calcularSiguienteTemperatura(T, Ta, k);
```

Se crea un frame con copias de T, Ta y k.

Paso por referencia

```
actualizarTemperatura(&T, Ta, k);
```

Se crea un frame con la dirección de T, por eso la función puede modificar la variable original.

Resultado esperado

Salida en consola

```
Paso 0 | 90.00 C | #####
Paso 1 | 84.80 C | #####
Paso 2 | 80.02 C | #####
Paso 3 | 75.62 C | #####
...
Paso 20 | 37.27 C | #####
```

Archivo generado

`datos_enfriamiento.csv`

Paso, Temperatura

0,90.00

1,84.80

2,80.02

3,75.62

Ejemplo: factorial

```
int fact(int n){  
    if(n <= 1) return 1;  
    return n * fact(n - 1);  
}
```

Idea clave

Una función puede llamarse a sí misma para resolver un problema en pasos más pequeños.

Ejemplo de evaluación

$$\text{fact}(4) = 4 * 3 * 2 * 1$$

¿Cuándo usar recursividad?

La recursividad es útil en problemas como:

- Factorial.
- Fibonacci.
- Árboles.
- Búsquedas.
- Algoritmos matemáticos.

Criterio práctico

Se recomienda cuando un problema se descompone naturalmente en subproblemas similares.

Separar el programa en archivos

- `main.c`
- `operaciones.c`
- `operaciones.h`

Ventajas

- Orden.
- Mantenimiento.
- Escalabilidad.
- Trabajo colaborativo.

Ejemplo modular

operaciones.h

```
int sumar(int, int);
```

operaciones.c

```
int sumar(int a, int b){  
    return a + b;  
}
```

main.c

```
printf("%d", sumar(2, 3));
```

Situaciones que debemos evitar

- No declarar el prototipo.
- Usar tipos incompatibles.
- No retornar un valor cuando corresponde.
- Confundir *p con p.
- Intentar usar variables fuera de su alcance.

Crear un programa que:

- 1 Lea dos números.
- 2 Use una función de suma.
- 3 Use una función de multiplicación.
- 4 Use un puntero para intercambiar valores.
- 5 Muestre los resultados.

Meta

Integrar funciones, parámetros y punteros en un mismo ejercicio.

Las funciones permiten escribir programas profesionales

- Funciones.
- Parámetros.
- Punteros.
- Variables locales y globales.
- Recursividad.
- Modularidad.

Reflexiona y responde

¿Por qué un programa dividido en funciones es mejor que uno de 500 líneas seguidas?

Modularidad = claridad + reutilización + mantenimiento